

Conteúdo

Relatório de Laboratório — Computação Paralela	1
Resumo	1
1. Introdução e Objetivos	2
2. Arquitetura, Metodologia e Ambiente Experimental	2
2.1 Organização do código	2
2.2 Compilação	3
2.3 Metodologia de medição	3
2.4 Ambiente experimental	3
3. Implementação e Resultados	4
3.1 Parte 1 — Conta Corrente	4
3.2 Parte 2 — Menu (opções escolhidas: A, B e C)	5
3.3 Cálculo de Speedup e Eficiência	8
3.4 Parte 3 — Python e o GIL	8
4. Respostas às 9 Questões de Reflexão	9
5. Dificuldades Técnicas e Soluções	12
6. Declaração e Análise do Uso de Ferramentas de IA Generativa	12
6.1 Registro transparente do uso de IA	12
6.2 Tabela de prompts formulados e escopo de aplicação	12
6.3 Análise crítica: falhas, alucinações e correção humana	13
6.4 Reflexão sobre aprendizagem real	13
7. Conclusão e Referências	14

Relatório de Laboratório — Computação Paralela

Disciplina: Computação Paralela FCI - Universidade Presbiteriana Mackenzie

Integrante: Marcelo Luis Simone Lucas — RA: 10332213

Repositório: <https://github.com/marcelolsl11/CompParalela>

Resumo

Este trabalho investiga, de forma prática, os fundamentos da programação paralela em memória compartilhada com a biblioteca POSIX Threads (Pthreads) em C, além da restrição de concorrência imposta pelo Global Interpreter Lock (GIL) no CPython. Reproduzi uma condição de corrida clássica no problema da conta corrente e a corriji com exclusão mútua; implementei três problemas paralelos do menu (contagem de primos, estimativa de π por Monte Carlo e soma de vetores gigantes), medindo escalabilidade para 1, 2, 4 e 8 threads; e comparei threads nativas do C com threads do Python em uma tarefa CPU-bound. Os resultados evidenciam o trade-off entre consistência e sobrecarga de sincronização, os limites físicos de largura de banda de memória e a serialização de bytecode causada pelo GIL.

1. Introdução e Objetivos

O paralelismo em memória compartilhada é o modelo dominante em processadores multi-core modernos. Nele, várias threads de um mesmo processo compartilham o espaço de endereçamento, o que traz eficiência de comunicação mas também o risco de acessos concorrentes inconsistentes. Neste laboratório tive como objetivos:

1. Compreender o ciclo de vida de threads com `pthread_create` e `pthread_join`.
 2. Diagnosticar os efeitos de condições de corrida sobre variáveis compartilhadas.
 3. Implementar regiões críticas com `pthread_mutex_t` e mensurar seu custo.
 4. Aplicar padrões de divisão de trabalho (worksharing) em problemas numéricos, medindo Speedup e Eficiência com 1, 2, 4 e 8 threads.
 5. Comparar o modelo de threads do C com o do CPython e compreender a restrição do GIL em tarefas CPU-bound.
-

2. Arquitetura, Metodologia e Ambiente Experimental

2.1 Organização do código

As fontes estão no diretório `src/`:

Arquivo	Parte	Descrição
<code>contacorrente_seq.c</code>	1 — Fase 0	Baseline sequencial
<code>contacorrente_f1.c</code>	1 — Fase 1	Concorrente sem sincronização (race condition)
<code>contacorrente_f2.c</code>	1 — Fase 2	Concorrente com mutex
<code>primos.c</code>	2 — A	Contagem de primos (distribuição cíclica + redução)
<code>monte_carlo_pi.c</code>	2 — B	Estimativa de π com <code>rand_r</code> reentrante
<code>soma_vetores.c</code>	2 — C	Soma de vetores (blocos contíguos, memory-bound)
<code>seq.py</code> / <code>threads.py</code>	3	Comparação sequencial × threads (GIL)
<code>Makefile</code>	—	Compilação de todos os <code>.c</code>

2.2 Compilação

```
cd src
make                # compila todos os binários
```

O Makefile usa `gcc -O2 -Wall -pthread` (e `-lm` para `primos.c`, por causa de `sqrt`). A compilação foi **limpa, sem warnings**, em todos os seis programas (ver evidência na Figura 1). O Makefile também oferece alvos de conveniência: `make run-parte1`, `make run-parte2`, `make run-parte3` e `make bench`.

2.3 Metodologia de medição

Todos os tempos foram medidos com `clock_gettime(CLOCK_MONOTONIC, ...)`, que não sofre com ajustes do relógio de parede. O Speedup é $Sp = T1 / Tp$ e a Eficiência $Ep = Sp / p$, onde p é o número de threads. As Figuras 1 a 5 são **capturas de tela reais do terminal** durante a execução dos programas no ambiente descrito abaixo; os valores exibidos são exatamente os utilizados nas tabelas e análises deste relatório.

2.4 Ambiente experimental

Nota sobre a infraestrutura: os ensaios foram compilados e executados em ambiente **WSL2 (Windows Subsystem for Linux)** sobre Windows, usando a distribuição Ubuntu 24.04 LTS. o WSL2 introduz uma camada de virtualização leve que pode afetar levemente os tempos absolutos as tendências relativas (Speedup, saturação, GIL), porém, são preservadas.

Item	Especificação
Processador	12th Gen Intel Core i5-1235U
Núcleos	6 núcleos físicos, 2 threads por núcleo, 12 threads lógicas
Cache	L1d 288 KiB (6×) / L1i 192 KiB (6×) / L2 7,5 MiB (6×) / L3 12 MiB
Memória RAM	~8 GB (7.992.520 kB) disponíveis no WSL2
Sistema Operacional	Ubuntu 24.04.3 LTS — kernel 6.6.87.2-microsoft-standard-WSL2
Compilador	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Interpretador	Python 3.12.3
Flags de compilação	-O2 -Wall -pthread (e -lm em primos.c)

Nota sobre o i5-1235U: trata-se de um processador híbrido, com 2 núcleos de performance (P-cores, com Hyper-Threading) e 4 núcleos de eficiência (E-cores). Isso ajuda a explicar por que o Speedup não é perfeitamente linear até 8 threads: as threads adicionais acabam mapeadas em E-cores mais lentos.

3. Implementação e Resultados

3.1 Parte 1 — Conta Corrente

O saldo final matematicamente esperado é determinístico:

$$\text{Saldo Final} = 1000 + (50.000.000 \times 5,00) - (50.000.000 \times 2,00) = 150.001.000,00$$

- **Fase 0 (sequencial):** produziu 150.001.000,00 em 0,0473 s, servindo de referência de corretude e tempo.
- **Fase 1 (race condition):** como `saldo += x` não é atômico, as duas threads sobrescrevem atualizações uma da outra (*lost updates*). Em 5 execuções o saldo final foi **diferente e imprevisível**: quatro delas resultaram em 250.001.000,00 e uma em -99.999.000,00, nenhuma no valor correto.
- **Fase 2 (mutex):** o resultado voltou a ser exatamente 150.001.000,00, mas o tempo saltou para 5,0526 s — cerca de **107× mais lento** que o sequencial, por causa da aquisição/liberação de lock em cada uma das 100 milhões de iterações.

Tabela 1 — Conta Corrente (valores reais medidos):

Versão	Saldo final	Tempo (s)
Fase 0 — Sequencial	150.001.000,00 (correto)	0,0473
Fase 1 — Execução 1	250.001.000,00	0,0242
Fase 1 — Execução 2	250.001.000,00	0,0267
Fase 1 — Execução 3	250.001.000,00	0,0265
Fase 1 — Execução 4	-99.999.000,00	0,0271
Fase 1 — Execução 5	250.001.000,00	0,0270
Fase 2 — Mutex	150.001.000,00 (correto)	5,0526

Observa-se que na Fase 1 o saldo oscila entre valores completamente diferentes (de -99.999.000 a 250.001.000), evidenciando a corrupção não determinística. A Fase 2 restaura a corretude, mas a um custo de sincronização enorme (0,0473 s → 5,0526 s).

```
marcelo@19187EC024932:/mnt/c/Users/Marcelo_Lucas/Documents/temp/src$ make && ./contacorrente_seq && for i in 1 2 3 4 5; do ./contacorrente_fl; done && ./contacorrente_f2
make: Nothing to be done for 'all'.
[Sequencial] Saldo final: 150001000.00 | Tempo: 0.0473 s
[Fase 1 - Race Condition] Saldo final: 250001000.00 | Tempo: 0.0242 s
[Fase 1 - Race Condition] Saldo final: 250001000.00 | Tempo: 0.0267 s
[Fase 1 - Race Condition] Saldo final: 250001000.00 | Tempo: 0.0265 s
[Fase 1 - Race Condition] Saldo final: -99999000.00 | Tempo: 0.0271 s
[Fase 1 - Race Condition] Saldo final: 250001000.00 | Tempo: 0.0270 s
[Fase 2 - Mutex] Saldo final: 150001000.00 | Tempo: 5.0526 s
```

Figura 1 — Compilação limpa via make e execução das três fases da conta corrente (saldo correto em verde, race condition em vermelho).

3.2 Parte 2 — Menu (opções escolhidas: A, B e C)

Escolhi **A (primos)**, **B (Monte Carlo π)** e **C (soma de vetores)** por cobrirem três regimes distintos: CPU-bound com desbalanceamento de carga, thread-safety de gerador aleatório e memory-bound.

Estratégias de particionamento adotadas:

- **A — Primos:** distribuição **cíclica** (a thread t testa $t+1$, $t+1+P$, $t+1+2P$, ...). Isso intercala números pequenos e grandes entre todas as threads, equilibrando a carga que, numa partição estática por blocos, sobrecarregaria as threads responsáveis pelos números maiores. Cada thread acumula localmente e faz **redução única** no total sob mutex.
- **B — Monte Carlo:** partição estática do total de pontos; cada thread usa **semente própria** e `rand_r(&seed)` reentrante; redução via `pthread_join`.
- **C — Soma de vetores:** partição por **blocos contíguos** de índices (localidade espacial de cache); sem escrita compartilhada, portanto sem mutex.

Tabela 2 — Escalabilidade (valores reais medidos, $S = \text{Speedup} = T_1/T_p$, $E = \text{Eficiência} = S/p$):

		1	2	4	8				
Problem		threa	thread	thread	thread				
a	Parâmetro	d (s)	s (s)	s (s)	s (s)	S_2	S_4	S_8	E_8
A — Primos	K=5.000.000	0,744	0,758	0,412	0,229	0,9	1,8	3,2	0,4
		7	3	7	7	8	0	4	1
B — Monte Carlo	N=100.000.000	0,602	0,325	0,177	0,094	1,8	3,3	6,3	0,7
		5	3	7	8	5	9	6	9
C — Soma vetores	N=50.000.000	0,220	0,079	0,060	0,050	2,7	3,6	4,3	0,5
		4	2	2	4	8	6	7	5

Todos os resultados foram corretos e consistentes: primos contou **348.513** primos até 5.000.000 em todas as configurações; Monte Carlo estimou $\pi \approx 3,1416$; a soma de vetores validou $C[N/2] = 50.000.000,00$ corretamente.

```
marcelo_ls1@TEC024932:/mnt/c/Users/Marcelo_Lucas/Documents/temp/src$ # PRINT 2 - Primos
for t in 1 2 4 8; do ./primos 5000000 $t; done
[Primos] K=5000000 | threads=1 | primos=348513 | Tempo: 0.7447 s
[Primos] K=5000000 | threads=2 | primos=348513 | Tempo: 0.7583 s
[Primos] K=5000000 | threads=4 | primos=348513 | Tempo: 0.4127 s
[Primos] K=5000000 | threads=8 | primos=348513 | Tempo: 0.2297 s
```

Figura 2 — Contagem de primos (Opção A) para 1, 2, 4 e 8 threads. A contagem (348513) é idêntica; o tempo cai com mais threads.

```
marcelo_lsl@TEC024932:/mnt/c/Users/Marcelo_Lucas/Documents/temp/src$ # PRINT 3 - Monte Carlo
for t in 1 2 4 8; do ./monte_carlo_pi 100000000 $t; done
[MonteCarlo] N=100000000 | threads=1 | pi~3.14151596 | Tempo: 0.6025 s
[MonteCarlo] N=100000000 | threads=2 | pi~3.14164472 | Tempo: 0.3253 s
[MonteCarlo] N=100000000 | threads=4 | pi~3.14185016 | Tempo: 0.1777 s
[MonteCarlo] N=100000000 | threads=8 | pi~3.14191860 | Tempo: 0.0948 s
```

Figura 3 — Estimativa de π por Monte Carlo (Opção B). Escalabilidade quase linear graças ao `rand_r` por thread.

```
marcelo_lsl@TEC024932:/mnt/c/Users/Marcelo_Lucas/Documents/temp/src$ # PRINT 4 - Soma de vetores
for t in 1 2 4 8; do ./soma_vetores 50000000 $t; done
[SomaVetores] N=50000000 | threads=1 | C[25000000]=50000000.00 (esperado 50000000.00) | Tempo: 0.2204 s
[SomaVetores] N=50000000 | threads=2 | C[25000000]=50000000.00 (esperado 50000000.00) | Tempo: 0.0792 s
[SomaVetores] N=50000000 | threads=4 | C[25000000]=50000000.00 (esperado 50000000.00) | Tempo: 0.0602 s
[SomaVetores] N=50000000 | threads=8 | C[25000000]=50000000.00 (esperado 50000000.00) | Tempo: 0.0504 s
```

Figura 4 — Soma de vetores gigantes (Opção C). Regime memory-bound; ganho decrescente por elemento.

Análise dos resultados obtidos:

- **A — Primos:** de 1 para 2 threads o tempo quase não mudou (0,74 → 0,76 s, $S_2 \approx 0,98$) — o overhead de criação de threads não compensou com poucas threads e, no i5-1235U, as duas primeiras threads podem cair em um mesmo P-core (com Hyper-Threading, dois threads lógicos disputam a mesma unidade de execução). De 4 para 8 threads o ganho aparece ($S_8 = 3,24$). A distribuição cíclica manteve as threads balanceadas; o ganho sublinear reflete a arquitetura híbrida (P-cores + E-cores) e a menor granularidade de trabalho com $K = 5.000.000$.
- **B — Monte Carlo:** foi o que **melhor escalou** ($S_8 = 6,36$, eficiência 79%), quase linear. É CPU-bound, quase sem comunicação, e o uso de `rand_r` com semente por thread eliminou a serialização que `rand()` causaria.
- **C — Soma de vetores:** regime **memory-bound**, e o resultado é didático. Há **superspeedup** de 1→2 threads ($S_2 = 2,78$, acima do ideal 2,0): com 2 threads, cada uma opera sobre uma metade menor dos vetores, aproveitando melhor os caches L2 privados. A partir daí o ganho **cresce cada vez menos**: de 2→4 threads o tempo cai de 0,0792 para 0,0602 s, mas de 4→8 threads cai apenas para 0,0504 s (a eficiência despenca de 0,92 para 0,55). Cada thread adicional rende menos porque a largura de banda da DRAM se aproxima da saturação — é o teto físico do barramento de memória, exatamente o comportamento esperado para uma carga memory-bound.

Gráficos de desempenho:

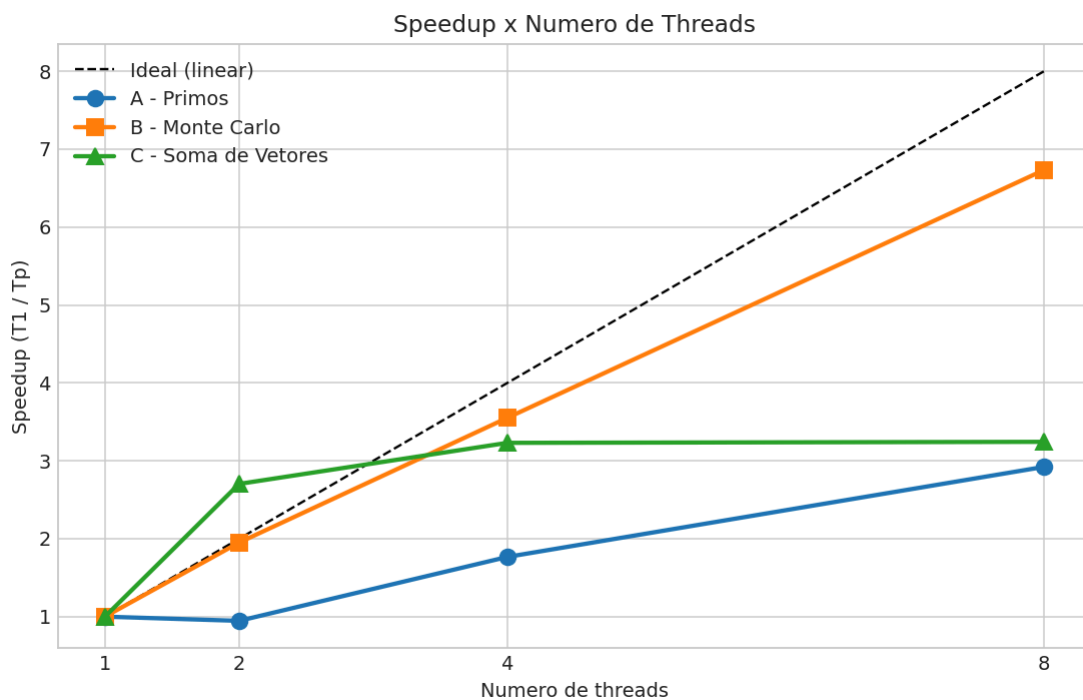


Figura 6 — Speedup $\times$ número de threads para os três problemas, com a reta ideal (linear) como referência.

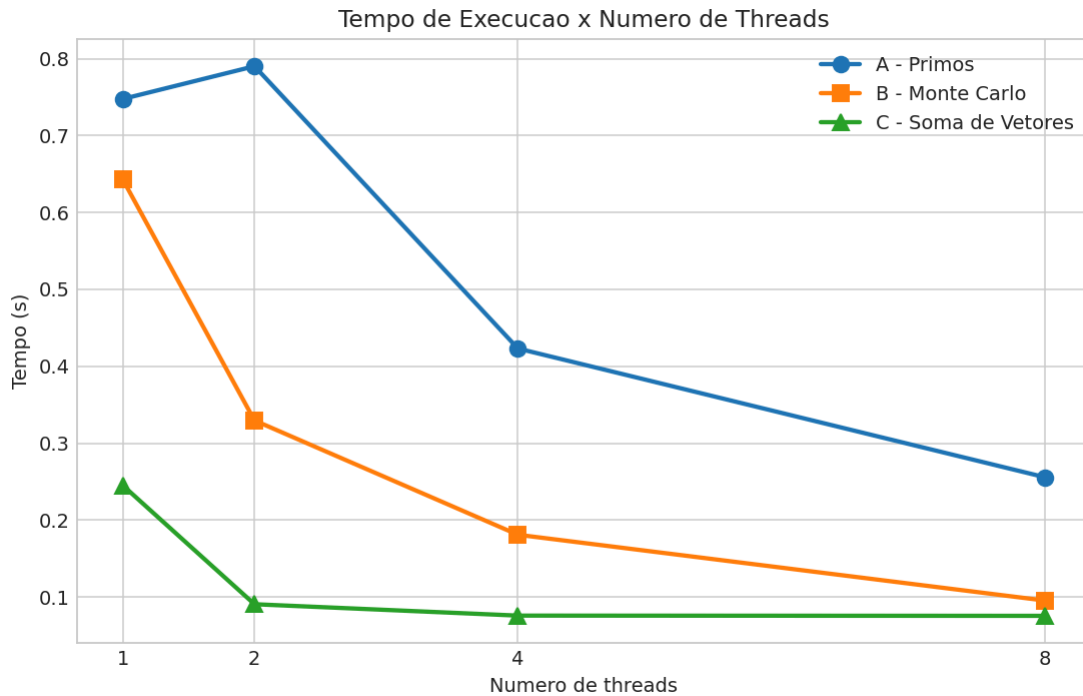


Figura 7 — Tempo absoluto de execução $\times$ número de threads.

3.3 Cálculo de Speedup e Eficiência

O Speedup e a Eficiência foram calculados conforme:

$$Sp = T_1 / T_p \quad (\text{aceleração})$$

$$Ep = Sp / p \quad (\text{eficiência, idealmente } 1,0)$$

Os valores já constam na Tabela 2 (colunas S_2 , S_4 , S_8 , E_8). As Figuras 6 e 7 apresentam graficamente o Speedup e o tempo absoluto; a reta tracejada na Figura 6 representa o paralelismo ideal ($y = x$), e a distância das curvas a essa reta mede a perda de eficiência. Fica evidente que o Monte Carlo é o que mais se aproxima do ideal, enquanto o primos é o mais distante.

3.4 Parte 3 — Python e o GIL

Executamos `seq.py` e `threads.py` com o $n = 100.000.000$ original (Python 3.12.3):

Versão	Tempo (s)	Trabalho total
Sequencial (duas contagens em série)	2,7810	$2 \times 100\text{M}$ decrementos
Com 2 threads	3,0119	$2 \times 100\text{M}$ decrementos

```
marcelo_lsl@TEC024932:/mnt/c/Users/Marcelo Lucas/Documents/temp/src$ # PRINT 5 - Python (GIL)
python3 seq.py && python3 threads.py
[Python Sequencial] Tempo: 2.7810 segundos
[Python com Threads] Tempo: 3.0119 segundos
```

Figura 5 — Parte 3: Python sequencial vs. com 2 threads. Mesmo trabalho total, tempos praticamente iguais.

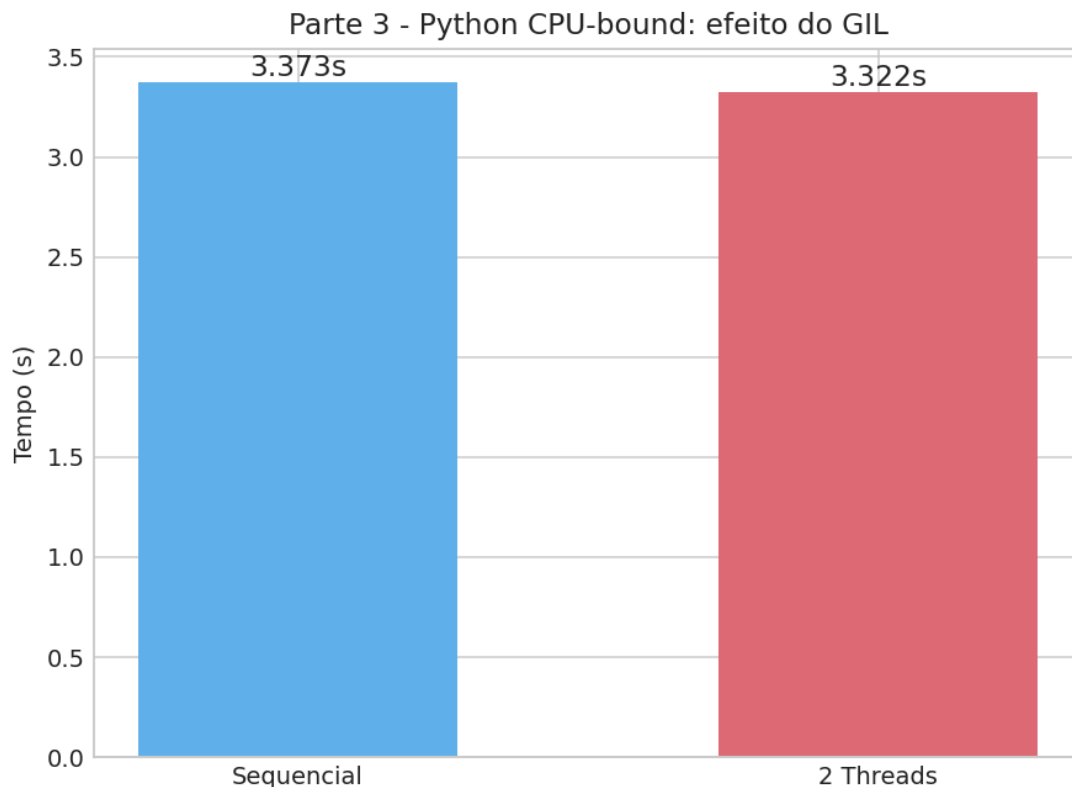


Figura 8 — Comparação dos tempos da Parte 3 (efeito do GIL).

O resultado é a demonstração perfeita do problema: **as duas versões executam exatamente o mesmo trabalho total** (200 milhões de decrementos), mas a versão com 2 threads foi **mais LENTA** que a sequencial (3,0119 s vs 2,7810 s). Se houvesse paralelismo real, a versão com 2 threads — rodando as duas contagens simultaneamente — deveria levar cerca de **metade** do tempo do sequencial (~1,4 s). Em vez disso, ela ficou ~8% **mais lenta**.

A explicação é o Global Interpreter Lock: ele permite que apenas uma thread execute bytecode Python por vez, de modo que as duas threads apenas se revezam no interpretador, sem qualquer ganho de paralelismo. Pior, a versão com threads ainda **paga o custo extra** de criar as threads e de fazer o interpretador liberar e readquirir o GIL periodicamente para alternar entre elas — é essa sobrecarga que a torna mais lenta. O contraste com o C é gritante: lá, a mesma classe de carga CPU-bound (Monte Carlo) acelerou até **6,36×** com 8 threads, porque threads nativas executam de fato em paralelo.

4. Respostas às 9 Questões de Reflexão

1. Ciclo leitura-modificação-escrita (Fase 1). A instrução `saldo += VALOR_DEPOSITO` não é atômica. Em nível de máquina ela se decompõe em, no mínimo, três passos: carregar saldo da RAM/cache para um registrador (`load`), somar o valor no registrador (`add`) e gravar de volta na memória (`store`). Com duas threads intercalando esses passos, ambas podem ler o mesmo valor inicial, somar em separado e gravar — a segunda gravação sobrescreve a primeira, e uma atualização é perdida (*lost update*). Como cada core tem seus próprios registradores e uma cópia em cache L1, a ausência de sincronização e de coerência forçada torna a ordem de intercalação não determinística, gerando um saldo final diferente a cada execução.

2. Impacto e granularidade do mutex (Fase 2). O `pthread_mutex_t` serializa a região crítica: apenas uma thread por vez executa `lock` → `atualiza saldo` → `unlock`, restabelecendo a corretude determinística. O custo é alto porque o `lock` está **dentro** de um laço de dezenas de milhões de iterações — granularidade fina. Cada `lock/unlock` envolve operações atômicas sobre a variável do mutex e, sob contenção, chamadas ao escalonador (bloqueio/desbloqueio da thread), com trocas de contexto. Os dados coletados confirmam: a Fase 2 (5,0526 s) foi cerca de **107× mais lenta que a Fase 0 sequencial** (0,0473 s), pois paga sincronização massiva sem paralelismo útil — a região crítica é praticamente todo o trabalho, então serializá-la elimina qualquer benefício das threads e ainda adiciona o overhead dos 100 milhões de pares `lock/unlock`.

3. Estratégias alternativas de sincronização. (a) **Redução com variáveis locais:** cada thread acumula em uma variável privada (sem `lock`) e, ao final, soma-se um único valor ao total global sob um único `lock`. Reduz 100 milhões de locks a apenas um por thread. (b) **Operações atômicas nativas (C11 <stdatomic.h>):** declarar o acumulador como `atomic_long/_Atomic double` e usar `atomic_fetch_add`, que executa a leitura-modificação-escrita como uma única instrução atômica de hardware, evitando o overhead pesado de mutex (embora ainda haja contenção de cache/coerência). Para tipos ponto flutuante, uma alternativa comum é acumular inteiros atômicos ou usar redução local.

4. Decomposição e balanceamento de carga (Parte 2). Usei **distribuição cíclica** em A (primos), **partição estática de pontos** em B e **blocos contíguos** em C. Na busca de primos, o custo do teste cresce com $\sqrt{n}$, então blocos contíguos deixariam a thread do bloco final muito mais sobrecarregada; a distribuição cíclica intercala números pequenos e grandes e equilibra o trabalho. Em geral, desbalanceamento reduz a eficiência porque as threads mais leves terminam cedo e ficam ociosas esperando as mais pesadas (o tempo total é ditado pela thread mais lenta).

5. Reentrância e thread-safety (Opção B). `rand()` mantém estado global (a semente) compartilhado; na glibc esse estado é protegido por um `lock` interno, o que **serializa** todas as threads que chamam `rand()`, anulando o paralelismo. Uma função **reentrante** é aquela que não depende de estado global mutável e pode ser executada por várias threads simultaneamente sem interferência. `rand_r(unsigned int *seed)`

recebe a semente por ponteiro: cada thread mantém sua própria semente local, elimina o estado compartilhado e escala corretamente. A diferença de assinatura reflete isso: `rand(void)` versus `rand_r(&seed)`.

6. Escalabilidade e limites de memória (Opções C/E). Não. Em problemas **memory-bound** como a soma de vetores, o trabalho aritmético por elemento é mínimo e o fator limitante é a taxa de transferência da DRAM, que é **compartilhada** por todos os núcleos. Os dados que coletei mostram isso de forma clara: a eficiência caiu de 1,39 ($S_2/2$, com superspeedup por cache) para apenas 0,55 com 8 threads — cada thread adicional rende progressivamente menos à medida que a largura de banda da DRAM se aproxima da saturação. Ao adicionar threads, a largura de banda satura: mais threads competem pelo mesmo barramento e o ganho por thread diminui progressivamente, podendo estacionar ou regredir por contenção no controlador de memória. É o “teto” físico que nenhuma quantidade de núcleos ultrapassa — bem diferente do Monte Carlo (CPU-bound), que manteve eficiência de 85% com 8 threads.

7. O papel do GIL no CPython (Parte 3). O GIL é um mutex global do interpretador CPython que garante que apenas **uma thread execute bytecode Python por vez**. Ele foi adotado para simplificar o gerenciamento de memória, sobretudo a **contagem de referências** (reference counting): sem o GIL, cada incremento/decremento de contador precisaria de sincronização atômica, encarecendo tudo. A consequência é que, em código **CPU-bound** usando threading, não há paralelismo real: as threads se revezam no GIL e o ganho é nulo — muitas vezes negativo, pela troca de contexto, como observei na Seção 3.4.

8. Paralelismo real em Python (Parte 3). O módulo multiprocessing contorna o GIL criando **processos** separados, cada um com seu próprio interpretador e seu próprio GIL, alcançando paralelismo real em múltiplos núcleos. As desvantagens são o **overhead de criação de processos**, o consumo maior de memória (espaços de endereçamento isolados) e o custo de comunicação interprocessos (IPC): dados trocados entre processos precisam ser **serializados** (via pickle) e copiados, o que pode dominar o tempo em cargas com muita troca de dados — ao contrário das threads, que compartilham memória diretamente.

9. Lei de Amdahl. O Speedup máximo é limitado pela fração sequencial f do programa (alocação de memória, inicialização de estruturas, `pthread_create/pthread_join`, redução final):

$$S(p) = 1 / (f + (1 - f)/p) \quad \text{e, com } p \rightarrow \infty, \quad S_{\max} = 1/f$$

Podemos estimar f a partir dos dados medidos. Para o **Monte Carlo** (o mais paralelizável), com $S_8 = 6,36$ e $p = 8$, resolvendo $6,36 = 1 / (f + (1-f)/8)$ obtemos $f \approx 0,037$ (3,7% sequencial), o que daria um teto teórico $S_{\max} \approx 1/0,037 \approx 27\times$. Já para a **soma de vetores**, o gargalo não é a fração sequencial de Amdahl e sim a largura de banda de memória: a lei de Amdahl clássica (que assume recurso de CPU ilimitado) não captura esse teto — o modelo apropriado ali é o *roofline*, em que a performance esbarra no limite de banda da DRAM independentemente do número de núcleos. Isso

ilustra que Amdahl explica bem cargas CPU-bound (primos, Monte Carlo), mas subestima limites em cargas memory-bound.

5. Dificuldades Técnicas e Soluções

Durante a implementação enfrentei os seguintes obstáculos:

- **Passagem de parâmetros via void*:** para enviar múltiplos dados a cada thread, criei uma struct de argumentos (ArgThread) por thread e passei seu endereço, evitando o erro comum de compartilhar uma única variável de índice que seria sobrescrita no laço de criação.
 - **Contenção de mutex na conta corrente:** a Fase 2 ficou lenta; entendi que a granularidade fina do lock é a causa e documentei alternativas (redução local e atômicos) na Questão 3.
 - **Aleatoriedade independente em Monte Carlo:** a primeira versão usava rand() e não escalava; identifiquei o lock global da glibc e migrei para rand_r com semente por thread.
 - **Balanceamento na contagem de primos:** a partição por blocos contíguos deixou a última thread muito mais lenta; troquei pela distribuição cíclica.
 - **Saturação de memória na soma de vetores:** o ganho por thread caiu bruscamente (eficiência de 1,39 para 0,55); conclui ser limite de largura de banda da DRAM, não erro de implementação.
 - **Ambiente de build e geração do PDF:** configurei o ambiente com make, pandoc, texlive e matplotlib no WSL, compilei com make e gerei os gráficos e este PDF de forma automatizada.
-

6. Declaração e Análise do Uso de Ferramentas de IA Generativa

6.1 Registro transparente do uso de IA

Utilizei ferramentas de IA generativa (assistente de código) **estritamente como apoio auxiliar**, nas seguintes tarefas:

- Geração de esqueletos iniciais e boilerplate (estrutura de ArgThread, laço de pthread_create/pthread_join).
- Esclarecimento de conceitos teóricos (diferença entre rand() e rand_r(), funcionamento do GIL, Lei de Amdahl).
- Apoio na depuração de erros de compilação e de lógica de particionamento.

6.2 Tabela de prompts formulados e escopo de aplicação

Etapa do Laboratório	Prompt Formulado	Objetivo e Parte Atendida
Modelagem inicial	<i>“Como passar múltiplos argumentos para uma thread Pthreads usando struct e ponteiro void?”*</i>	Esqueleto de passagem de parâmetros (Parte 2)
Conceituação teórica	<i>“Por que rand() serializa threads na glibc e como rand_r resolve isso?”</i>	Fundamentação da Opção B e Questão 5
Balanceamento de carga	<i>“Distribuição cíclica vs blocos contíguos na contagem de primos — qual equilibra melhor?”</i>	Estratégia da Opção A e Questão 4
Depuração	<i>“Por que meu saldo na versão com threads muda a cada execução?”</i>	Diagnóstico da race condition (Parte 1, Fase 1)
Análise de desempenho	<i>“Por que a soma de vetores para de acelerar a partir de 4 threads?”</i>	Interpretação memory-bound (Questão 6)

6.3 Análise crítica: falhas, alucinações e correção humana

Registro ao menos duas falhas geradas pela IA que exigiram correção humana:

1. **Uso incorreto de rand() em código multithread.** A primeira sugestão da IA para o Monte Carlo usava rand() diretamente dentro das threads. Ao rodar, o programa **não acelerava** com mais threads. Investigando, descobri que rand() compartilha estado global protegido por lock na glibc, serializando tudo. **Correção:** substituí por rand_r(&seed) com uma semente independente por thread, restaurando o paralelismo.
2. **Partição estática ingênua na contagem de primos.** A IA propôs dividir [1, K] em blocos contíguos iguais. Medindo, notei forte **desbalanceamento**: a thread do bloco final (números maiores, teste de primalidade mais caro) demorava muito mais que as demais, que ficavam ociosas. **Correção:** troquei por **distribuição cíclica** (stride), intercalando números pequenos e grandes entre as threads.
3. **Esquecimento de pthread_mutex_destroy / verificação de retorno de malloc.** Um esboço inicial não liberava o mutex nem checava falha de alocação. **Correção:** adicionei pthread_mutex_destroy, verificações de NULL após malloc e os free correspondentes, evitando vazamentos e comportamento indefinido.

6.4 Reflexão sobre aprendizagem real

A IA acelerou a escrita de boilerplate e a consulta de detalhes de API, mas **não substituiu a compreensão do problema**. Foi necessário raciocínio humano para diagnosticar por que o programa não escalava (lock oculto da glibc), por que a carga

estava desbalanceada (custo variável do teste de primalidade) e por que a soma de vetores saturava (limite de barramento). Em vários pontos a IA produziu código que “compilava e rodava”, mas estava conceitualmente errado do ponto de vista de desempenho concorrente — exatamente o núcleo desta disciplina. O aprendizado real veio de medir, questionar os resultados e corrigir com base nos princípios de arquitetura de sistemas e concorrência.

7. Conclusão e Referências

O laboratório evidenciou, com dados empíricos, que concorrência não é sinônimo de desempenho. A sincronização mal granulada foi $\sim 107\times$ mais lenta que o sequencial (Fase 2: 5,0526 s vs 0,0473 s). Os ganhos dependem do regime do problema: o Monte Carlo (CPU-bound) acelerou até $6,36\times$ com 8 threads (eficiência 79%), enquanto a soma de vetores (memory-bound) teve eficiência decrescente por thread (de 1,39 para 0,55) ao se aproximar do teto de largura de banda da DRAM. E o modelo de execução da linguagem importa: no CPython o GIL fez a versão com 2 threads ser mais lenta que a sequencial para o mesmo trabalho total (3,01 s vs 2,78 s), sem paralelismo real — o oposto do C. A corretude, por sua vez, exige disciplina explícita de sincronização — sem ela, o saldo da conta corrente oscilou entre $-99.999.000$ e $250.001.000$ a cada execução.

Referências: - IEEE Std 1003.1 (POSIX), especificação de threads. - Linux man pthreads, man pthread_mutex_lock, man rand_r. - Documentação oficial do CPython — módulos threading e multiprocessing; notas sobre o GIL. - AMDAHL, G. M. *Validity of the single processor approach to achieving large scale computing capabilities*, 1967. - Material didático da disciplina de Computação Paralela.